

第七章：同步原语

一、核心概念

1.1 竞态条件 (Race Condition)

当多个执行流（多核 CPU、中断处理函数）并发访问共享数据，且至少一个是写操作时，如果没有适当的同步，结果取决于执行顺序——这就是竞态条件。

```
CPU 0: read counter (=5)
CPU 1: read counter (=5)
CPU 0: write counter = 5+1 = 6
CPU 1: write counter = 5+1 = 6
// 期望 counter=7, 实际 counter=6 → 丢失更新
```

1.2 临界区 (Critical Section)

访问共享数据的代码段称为临界区。同步原语的目的就是保证临界区的**互斥**（同一时刻只有一个执行流在临界区内）。

1.3 自旋锁 (Spinlock)

最基本的锁。等不到锁的 CPU 忙等（自旋）。

原子操作：锁的获取必须是原子的——“检查锁是否空闲”和“占有锁”不能被打断。硬件提供原子指令：

- RISC-V: `amoswap` (atomic swap)
- x86: `lock xchg` / `lock cmpxchg`
- ARM: `ldaxr/stlxr` (LL/SC)

内存屏障：多核处理器可能重排内存操作。锁的获取/释放需要内存屏障确保临界区内的操作不会被重排到临界区外。

1.4 睡眠锁 (Sleep Lock / Mutex)

自旋锁持锁期间不能做任何可能阻塞的操作（磁盘 I/O、sleep），且自旋浪费 CPU。

睡眠锁的方案：等不到锁就让出 CPU（sleep），锁释放时唤醒（wakeup）。

1.5 sleep/wakeup 机制

操作系统中的条件等待原语：

- `sleep(channel, lock)`：释放锁，将当前进程标记为 SLEEPING，让出 CPU
- `wakeup(channel)`：唤醒所有在该 channel 上等待的进程

丢失唤醒 (Lost Wakeup) 问题：

```
// 错误写法：
release(lock);           // ← wakeup 可能在这里发生！
sleep(channel);          // ← 已经错过 wakeup，永远睡死

// 正确写法：
sleep(channel, lock); // 在持有 p->lock 的保护下原子地释放 lock + 设 SLEEPING
```

1.6 更高级的同步原语

原语	思想	Linux 对应
信号量 (Semaphore)	计数器，允许 N 个执行流同时进入	struct semaphore
读写锁 (RWLock)	读共享、写互斥	rwlock_t / rw_semaphore
RCU (Read-Copy-Update)	读无锁，写时复制后替换指针	Linux 内核大量使用
完成量 (Completion)	等待某个事件完成	struct completion
Futex	用户态快速路径 + 内核慢路径	futex() syscall

二、基本推论

- 推论 1：自旋锁必须关中断。** 如果持锁时被中断打断，中断处理函数又尝试获取同一把锁 → 死锁。因此 acquire 前必须关闭本核中断。
- 推论 2：持自旋锁时不能睡眠。** 如果持锁的 CPU 让出执行权 (sleep/yield)，其他等待这把锁的 CPU 会一直自旋。更严重的是，被唤醒的进程可能在另一个 CPU 上运行，导致锁的 CPU 亲和性假设失效。
- 推论 3：sleep/wakeup 需要第二把锁来防止丢失唤醒。** sleep 必须在释放条件锁和设置 SLEEPING 之间保持某种原子性。xv6/RUOS 的做法是用 p->lock 提供这个原子性。
- 推论 4：锁的粒度决定并发度。** 一把全局大锁简单但串行化所有操作。细粒度锁 (per-inode, per-buffer) 提高并发度但增加死锁风险和代码复杂度。
- 推论 5：死锁的四个必要条件。** (1) 互斥 (2) 持有并等待 (3) 不可剥夺 (4) 循环等待。打破任一条件即可避免死锁。实践中常用固定加锁顺序 (打破循环等待)。

三、Linux / 通用实现

3.1 Linux spinlock

```
// 基于 ticket 的自旋锁 (保证公平性, FIFO 顺序)
typedef struct {
    union {
        u32 slock;
        struct {
            u16 owner; // 当前服务号
            u16 next;  // 下一个等待号
        };
    };
};
```

```

        };
    };
} arch_spinlock_t;

lock:  my_ticket = atomic_fetch_add(&next, 1)
        while (owner != my_ticket) cpu_relax()

unlock: owner++

```

Linux 4.2+ 在 x86 上用 **queued spinlock (qspinlock)**——更好的 NUMA 性能。

3.2 Linux mutex

```

struct mutex {
    atomic_long_t owner;    // 持有者 task_struct 指针
    struct optimistic_spin_queue osq; // 乐观自旋队列
    struct list_head wait_list;      // 等待者链表
};

```

三阶段获取：

1. 快速路径：cmpxchg 尝试直接获取（无竞争时最快）
2. 乐观自旋：如果持有者在其他 CPU 上运行，短暂自旋等待
3. 慢速路径：放入 wait_list, sleep

3.3 RCU (Read-Copy-Update)

Linux 内核最重要的同步机制之一：

- **读**：无锁，只用 `rcu_read_lock()`（本质是关抢占）
- **写**：复制数据 → 修改副本 → 原子替换指针 → 等待所有读者完成（grace period）→ 释放旧数据

适用于读多写少的场景（路由表、进程列表等）。

3.4 Futex

Fast Userspace Mutex：

- **无竞争**：纯用户态原子操作，不陷入内核
- **有竞争**：通过 `futex(FUTEX_WAIT)` 系统调用进入内核等待
- 是 `pthread_mutex` 的底层实现

四、RUOS 的实现

核心文件

- `src/sync/spinlock.h / spinlock.c` — 自旋锁
- `src/sync/sleeplock.h / sleeplock.c` — 睡眠锁

4.1 spinlock: 关中断 + 原子 TAS

```
void acquire(struct spinlock *lk) {
    push_off();           // 关中断（可嵌套，引用计数）
    while (__sync_lock_test_and_set(&lk->locked, 1) != 0)
        ;                // 自旋
    __sync_synchronize(); // 内存屏障
    lk->cpu = mycpu();
}

void release(struct spinlock *lk) {
    lk->cpu = 0;
    __sync_synchronize();
    __sync_lock_release(&lk->locked);
    pop_off();           // 恢复中断
}
```

push_off/pop_off 嵌套计数:

```
push_off():
    old = intr_get();
    intr_off();
    if (cpu->noff == 0) cpu->intena = old; // 记录最外层的中断状态
    cpu->noff++;

pop_off():
    cpu->noff--;
    if (cpu->noff == 0 && cpu->intena)
        intr_on(); // 只有最外层且之前是开中断的，才恢复
```

对比 Linux: Linux 的 `spin_lock_irqsave()` 也是关中断 + 保存旧状态，思路一致。区别在于 Linux 用 ticket/queued spinlock 保证公平性，RUOS 用简单 TAS（可能不公平）。

4.2 sleeplock

```
struct sleeplock {
    uint locked;
    struct spinlock lk;
    int pid;
};

void acquiresleep(struct sleeplock *lk) {
    acquire(&lk->lk);
    while (lk->locked)
        sleep(lk, &lk->lk); // 等不到就睡
    lk->locked = 1;
    lk->pid = myproc()->pid;
    release(&lk->lk);
}
```

```
    }

    void releasesleep(struct sleeplock *lk) {
        acquire(&lk->lk);
        lk->locked = 0;
        lk->pid = 0;
        wakeup(lk);
        release(&lk->lk);
    }
```

典型场景：inode 锁 (*ilock/iunlock*) —— 持锁期间需要读磁盘，不能用 spinlock。

4.3 sleep/wakeup

```
void sleep(void *chan, struct spinlock *lk) {
    struct proc *p = myproc();
    acquire(&p->lock);      // 先拿 p->lock
    release(lk);            // 再释放条件锁 (保证不丢失 wakeup)
    p->chan = chan;
    p->state = SLEEPING;
    sched();               // 切到调度器
    p->chan = 0;
    release(&p->lock);
    acquire(lk);           // 醒来后重新获取条件锁
}

void wakeup(void *chan) {
    for (p in proc[]) {
        acquire(&p->lock);
        if (p->state == SLEEPING && p->chan == chan)
            p->state = RUNNABLE;
        release(&p->lock);
    }
}
```

wakeup 的 O(n) 遍历：每次 wakeup 遍历所有 64 个 proc 槽。Linux 用 wait_queue 链表，只唤醒等待特定 channel 的进程。

4.4 锁的使用规范

场景	锁类型	原因
进程状态	p->lock (spin)	短临界区，多核竞争
文件表分配	ftable.lock (spin)	快速操作
inode 操作	ip->lock (sleep)	可能涉及磁盘 I/O
块缓存	bcache.lock + buf.lock	全局查找用 spin，单个 buf 用 sleep
管道	pi->lock (spin)	保护环形缓冲区

场景	锁类型	原因
eventfd	efd->lock (spin)	保护 counter

五、八股 / 面试高频问题

- Q: 自旋锁和互斥锁的区别？什么时候用哪个？** A: 自旋锁忙等，适合临界区很短（几个指令）的场景，不能睡眠。互斥锁（mutex）会让出 CPU，适合临界区较长或可能阻塞的场景。在中断上下文只能用自旋锁。
- Q: 为什么自旋锁要关中断？** A: 防止本核中断处理函数尝试获取同一把锁导致死锁。多核系统中，关中断只关本核的，其他核的中断不受影响。
- Q: 什么是死锁？如何避免？** A: 两个或多个执行流互相等待对方持有的锁。避免方法：(1) 固定加锁顺序 (2) 加锁超时 (3) 避免嵌套锁 (4) trylock 非阻塞尝试。Linux 的 lockdep 工具可以动态检测死锁。
- Q: 什么是 RCU？适用什么场景？** A: Read-Copy-Update，读无锁，写时复制替换。适用于读多写少的场景（路由表、/proc 数据）。读者零开销，写者承担复制和 grace period 等待的代价。
- Q: Linux 的 futex 是什么？为什么需要它？** A: Futex 在无竞争时完全在用户态完成（原子操作），只有竞争时才陷入内核。这避免了每次 mutex lock/unlock 都做系统调用的开销。
- Q: 内存屏障是什么？为什么需要？** A: CPU 和编译器可能重排内存操作以优化性能。内存屏障指令（fence/mfence/dmb）强制保证屏障前后的内存操作不被重排。锁的 acquire 需要 acquire barrier，release 需要 release barrier。

六、项目贡献亮点

- 实现了嵌套安全的 spinlock（push_off/pop_off 引用计数），理解关中断与锁的关系
- sleep/wakeup 机制正确防止了丢失唤醒问题，是管道、eventfd、inode 等阻塞操作的基础

七、设计取舍总结

决策	RUOS 选择	Linux 做法	权衡
spinlock	TAS（不保证公平）	ticket/queued（FIFO 公平）	简单，2 核下问题不大
中断处理	push_off/pop_off 嵌套	spin_lock_irqsave	思路一致
睡眠同步	sleeplock + sleep/wakeup	mutex + wait_queue	channel-based vs queue-based
wakeup	O(n) 遍历全部进程	wait_queue 链表	64 个进程下可接受
Futex	仅基本 FUTEX_WAIT/WAKE	完整 futex 实现	满足 clone 的需要